

Password toolkit

Nuestra herramienta no solo sirve para generar y medir contraseñas. Ahora también puede crackear contraseñas por fuerza bruta y por diccionario, revisar si una contraseña ya fue filtrada en internet, generar frases de contraseña fáciles de recordar y hasta usar un ritmo de aplausos como contraseña. A continuación, se documentan las librerías nuevas que se usaron y como se desarrolló cada funcionalidad.

Horas dedicadas al desarrollo del proyecto: aproximadamente 30 horas.

tecnologías utilizadas:

```
#include <string_view>
```

string_view es una herramienta de C++ que permite mirar una cadena de texto que ya existe en memoria sin copiarla. En vez de ser dueño de los datos, solo guarda un puntero al inicio y el tamaño. Esto sirve para recorrer archivos gigantes como rockyou.txt sin tener que crear millones de strings nuevos, ya que cada copia sería una reserva de memoria y eso es lento.

Uso en el Código:

```
string_view datos(buffer.data(), buffer.size());  
  
string_view linea = datos.substr(inicio, fin - inicio);
```

Aquí se crea un string_view sobre el buffer que tiene todo el archivo cargado, y con substr se va sacando cada línea sin copiar nada, solo apuntando a una parte distinta del mismo bloque de memoria.

```
#include <vector>
```

vector es un arreglo dinámico de C++ que puede crecer según se necesite. Se usa para guardar el archivo completo en memoria como bytes y también para guardar la lista de palabras del generador de frases.

Uso en el código:

```
vector<char> buffer(tamano);  
archivo.read(buffer.data(), tamano);
```

Aquí se reserva un solo bloque de memoria del tamaño exacto del archivo y se lee todo de golpe. Una sola reserva grande es mucho más rápida que miles de reservas pequeñas.

```
ifstream archivo(nombre, ios::binary | ios::ate);
```

Para leer el diccionario se abre el archivo en modo binario para que no se transformen los bytes, y con la bandera ate el cursor se coloca al final. Así se puede saber el tamaño total del archivo con tellg antes de leerlo.

Uso en el código:

```
streamsize tamano = archivo.tellg();  
archivo.seekg(0, ios::beg);
```

Como el cursor estaba al final, su posición es igual al tamaño del archivo en bytes. después se regresa el cursor al inicio con seekg para empezar a leer desde el principio.

```
#include <chrono>
```

chrono es la librería de C++ para medir tiempo. Se uso para mostrarle al usuario cuanto tarda el programa en crackear una contraseña, lo cual sirve para que vea que tan vulnerable es.

Uso en el código:

```
auto inicio_t = chrono::high_resolution_clock::now();  
// ... proceso de crackeo ...  
auto fin_t = chrono::high_resolution_clock::now();  
double segundos = chrono::duration<double>(fin_t - inicio_t).count();
```

Se toma una marca de tiempo antes y otra después del proceso, y la diferencia entre ambas da los segundos que tardo, expresados como numero decimal.

```
#include <portaudio.h>  
#include <thread>
```

PortAudio es una librería externa que permite acceder al micrófono y reproducir audio desde C++ y funciona en varios sistemas operativos. thread es la librería de C++ para manejar hilos, es decir, hacer que el programa haga dos cosas a la vez. Estas dos se usaron para la funcionalidad de contraseña por ritmo: PortAudio escucha el micrófono en segundo plano mientras el programa principal lleva el ritmo del metrónomo.

Uso en el Código:

```
Pa_Initialize();
```

```
PaStream* stream;
Pa_OpenDefaultStream(&stream, 1, 0, paFloat32,
    SAMPLE_RATE, FRAMES, callback, nullptr);
Pa_StartStream(stream);
```

Se inicializa PortAudio, se abre el micrófono por defecto con un canal de entrada, y se le pasa una función callback que se ejecuta automáticamente cada vez que llega audio nuevo. Mientras tanto el programa sigue corriendo el metrónomo.

sección de desarrollo:

Crackeo por fuerza bruta =

La primera forma de crackear es la fuerza bruta, que consiste en probar todas las combinaciones posibles de caracteres hasta encontrar la contraseña. Se hizo con recursión: la función se llama a si misma agregando una letra cada vez.

Uso en el código:

```
bool Cracker::Brute(string& c_objetivo, string actual,
    short max_longitud) {
    if (c_objetivo == actual) {
        cout << "La contraseña es: " << actual << endl;
        return true;
    }
    if (actual.length() >= max_longitud) {
        return false;
    }
    for (char letra : letras) {
        if (Brute(c_objetivo, actual + letra, max_longitud)) {
            return true;
        }
    }
    return false;
}
```

La función hace tres cosas en orden. Primero compara si el texto que va armando es igual a la contraseña objetivo, si lo es la encontró y avisa. Si no, revisa si ya llego al largo máximo, en cuyo caso corta esa rama para no seguir infinitamente. Si todavía hay espacio, recorre cada letra del alfabeto, le pega una al texto actual y se vuelve a llamar a si misma. De esta manera prueba primero "a", luego "aa", "aaa", y así hasta agotar combinaciones o encontrar la contraseña.

Desde el main se va incrementando el largo máximo de uno en uno, empezando en 1, para encontrar primero las contraseñas cortas sin tener que probar las largas. También se mide el tiempo que tarda con chrono para mostrárselo al usuario.

Crackeo por diccionario =

La fuerza bruta tarda muchísimo con contraseñas largas, por eso también se hizo un crackeo por diccionario, que en vez de probar todas las combinaciones, compara la contraseña contra una lista enorme de contraseñas reales filtradas (rockyou.txt, que tiene alrededor de catorce millones).

El primer paso es leer todo el archivo de un solo golpe a memoria, porque abrir y leer línea por línea catorce millones de veces sería lentísimo.

Uso en el código:

```
vector<char> leerArchivoCompleto(const string& nombre) {
    ifstream archivo(nombre, ios::binary | ios::ate);
    if (!archivo) {
        throw runtime_error("No se pudo abrir el archivo");
    }
    streamsize tamano = archivo.tellg();
    archivo.seekg(0, ios::beg);
    vector<char> buffer(tamano);
    if (!archivo.read(buffer.data(), tamano)) {
        throw runtime_error("Error al leer el archivo");
    }
    return buffer;
}
```

Esta función abre el archivo, mide su tamaño, reserva un buffer de ese tamaño exacto y lee todo de una sola vez. Si algo falla al abrir o al leer, lanza una excepción que se atrapa más arriba para avisar del error sin que el programa se caiga.

Una vez cargado el archivo en memoria, se recorre línea por línea comparando cada una con la contraseña buscada, usando string_view para no copiar nada.

Uso en el código:

```
bool comprobarLineas(string_view datos, string_view objetivo) {
    size_t inicio = 0;
    size_t fin;
    while ((fin = datos.find('\n', inicio)) != string_view::npos) {
        string_view linea = datos.substr(inicio, fin - inicio);
        if (linea == objetivo) {
            cout << "Contraseña encontrada: " << linea << endl;
            return true;
        }
        inicio = fin + 1;
    }
    return false;
}
```

```

    }
    inicio = fin + 1;
}
return false;
}

```

Se coloca un índice al inicio del buffer y se busca el siguiente salto de línea. Lo que hay entre el inicio y ese salto es una contraseña del diccionario, y se compara con la que se busca. Si coincide se encontró, si no se mueve el índice a la siguiente línea y se repite. Como se usa `string_view` en vez de strings normales, no se hace ni una sola copia en todo el recorrido, lo que lo hace muy rápido a pesar del tamaño del archivo.

Verificar si una contraseña fue filtrada =

Esta funcionalidad reutiliza la misma lógica del diccionario, pero en vez de pedir la contraseña por consola, la recibe directamente como argumento. Sirve para revisar rápido si una contraseña propia ya aparece en filtraciones.

Uso en el Código:

```

bool verificarPwned(const string& contrasena) {
    vector<char> datos = leerArchivoCompleto("rockyou.txt");
    string_view contenido(datos.data(), datos.size());
    return comprobarLineas(contenido, contrasena);
}

```

Carga el archivo, crea un `string_view` sobre el y se lo pasa a `comprobarLineas`. Si la contraseña esta en la lista, le avisa al usuario que ya fue filtrada y que no debería usarla.

Generador de frases de contraseña =

Las contraseñas aleatorias como `k8$Rz` son seguras pero imposibles de recordar. Por eso se agrego un generador de frases tipo diceware, que elige varias palabras al azar de una lista y las une con guiones, dando algo como `mesa-rojo-campo-luz`, que es fácil de recordar y aun así difícil de adivinar.

Uso en el código:

```

string GenerarPassphrase(int num_palabras) {
    vector<char> buffer = leerArchivoCompleto("wordlist.txt");
    string_view datos(buffer.data(), buffer.size());
    vector<string> palabras;
    size_t inicio = 0, fin;
    while ((fin = datos.find('\n', inicio)) != string_view::npos) {
        palabras.push_back(string(datos.substr(inicio, fin - inicio)));
        inicio = fin + 1;
    }
}

```

```

    }
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<int> dist(0, palabras.size() - 1);
    string resultado;
    for (int i = 0; i < num_palabras; i++) {
        if (i > 0) resultado += "-";
        resultado += palabras[dist(gen)];
    }
    return resultado;
}

```

Primero se carga la lista de palabras igual que el diccionario y se mete cada palabra en un vector. Luego se prepara el generador de números aleatorios con mt19937, y se eligen tantas palabras al azar como pidió el usuario, pegándolas con guiones entre cada una. El programa pregunta por consola cuantas palabras quiere antes de generarla.

Contraseña por ritmo de aplausos =

La funcionalidad más distinta del proyecto es usar un ritmo como contraseña. El programa reproduce un metrónomo y el usuario aplaude (o presiona una tecla) en los tiempos que quiera. Ese patrón de golpes y silencios se convierte en una llave que después hay que repetir para acceder.

Para escuchar el micrófono se usa PortAudio con una función callback que se ejecuta sola cada vez que llega audio. Esta función revisa que tan fuerte fue el sonido y, si pasa cierto umbral, marca que hubo un aplauso.

Uso en el Código:

```

static int callback(const void* input, void*, unsigned long frames,
    const PaStreamCallbackTimeInfo*, PaStreamCallbackFlags, void*) {
    const float* samples = (const float*) input;
    if (!samples) return paContinue;
    float volumen_max = 0;
    for (unsigned long i = 0; i < frames; i++) {
        float v = fabs(samples[i]);
        if (v > volumen_max) volumen_max = v;
    }
    if (volumen_max > UMBRAL) aplauso_detectado = true;
    return paContinue;
}

```

El audio llega como números entre menos uno y uno, donde el silencio es cercano a cero y un aplauso fuerte se acerca al uno. Se busca el valor mas fuerte del pedazo de audio

usando valor absoluto, porque importa la magnitud y no si la onda va hacia arriba o hacia abajo. Si ese valor supera el umbral, se marca que hubo un aplauso.

El patrón se captura dividiendo cada beat en dos para poder detectar también golpes a contratiempo. El metrónomo suena en cada beat y, después de esperar el tiempo correspondiente, se revisa si el callback marco un aplauso, formando una cadena de unos y ceros.

Uso en el código:

```
for (int i = 0; i < subdivisiones; i++) {
    aplauso_detectado = false;
    if (i % 2 == 0) {
        Beep(1000, 100);
        cout << "Beat " << (i/2 + 1) << "/" << num_beats << endl;
    }
    this_thread::sleep_for(chrono::milliseconds(ms_por_sub));
    if (aplauso_detectado) patron += "1";
    else patron += "0";
}
```

Para verificar el acceso se compara el patrón guardado con el que se acaba de tocar, contando cuantas posiciones coinciden. Como nadie aplaude exactamente igual dos veces, no se exige una coincidencia perfecta sino un porcentaje mínimo, y se dan tres intentos.

Uso en el código:

```
int RitmoCracker::ContarCoincidencias(const string& a,
                                     const string& b) {
    int coincidencias = 0;
    for (int i = 0; i < (int)a.length(); i++) {
        if (a[i] == b[i]) coincidencias++;
    }
    return coincidencias;
}
```

Esta función compara posición por posición los dos patrones y cuenta cuantos unos y ceros coinciden. Si el número de coincidencias llega al mínimo establecido, se concede el acceso. también se hizo una versión alternativa que en vez de micrófono usa la barra espaciadora, para los casos en los que el sistema no tiene acceso al micrófono, como dentro de una máquina virtual.

Portabilidad entre sistemas =

Como el proyecto se desarrolló en Windows pero debe correr en un sistema Unix, se usó compilación condicional para que el mismo código funcione en ambos. En Windows se usa la función Beep del sistema para el metrónomo y PortAudio para el micrófono; en Linux, donde Beep no existe, se usa una alternativa. Esto se logra con ifdef, que revisa al momento de compilar en que sistema operativo se esta.

Uso en el código:

```
#ifdef _WIN32
    Beep(frecuencia, duracion_ms);
#else
    // alternativa para Linux
#endif
```

Cuando se compila en Windows, el compilador solo incluye la parte de Beep. Cuando se compila en Linux, ignora esa parte e incluye la alternativa. Así el mismo archivo de código sirve para los dos sistemas sin tener que cambiar nada a mano.

Resumen del proyecto:

¿Que resuelve el proyecto?

La herramienta permite generar contraseñas seguras, generar frases de contraseña fáciles de recordar, medir la fuerza de una contraseña, crackear contraseñas por fuerza bruta y por diccionario para demostrar que tan vulnerables son, revisar si una contraseña ya fue filtrada en internet, y además ofrece una forma experimental de autenticación basada en un ritmo de aplausos.

¿A quien va dirigido el proyecto?

Va dirigido a usuarios que quieren mejorar su seguridad digital, como personas que necesitan crear y verificar contraseñas seguras, y también a estudiantes o curiosos que quieran entender de forma práctica cómo funcionan los ataques de fuerza bruta y de diccionario, y por qué ciertas contraseñas son tan fáciles de romper.

Razón de ser del proyecto

El proyecto nació con la idea de hacer un crackeador de contraseñas por diversión y para aprender. En la primera versión solo generaba y media contraseñas, pero conforme avanzo se fue agregando los crackeadores, la verificación de filtraciones y el generador de frases. La parte del ritmo se agregó para experimentar con audio y con una forma de

autenticación distinta a la tradicional. La razón principal fue aprender sobre seguridad de contraseñas, manejo de memoria y archivos grandes en C++, y trabajar con librerías externas como PortAudio.

¿Por qué usar estas librerías?

Se uso `string_view` y `vector` para poder cargar y recorrer un diccionario de catorce millones de contraseñas sin que el programa se volviera lentísimo por estar copiando datos. `random` se uso por dar números aleatorios mas confiables que `rand` para la generación de contraseñas. `chrono` se uso para medir y mostrar el tiempo de crackeo. Y `PortAudio` junto con `thread` se usaron porque eran la forma de acceder al micrófono y escuchar en segundo plano mientras el programa lleva el ritmo, que era lo que necesitaba la funcionalidad de contraseña por ritmo.